A Degree-Constrained Algebraic Relaxation Heuristic for the Euclidean Traveling Salesperson Problem

Spectral Connectivity Regularization, Binary Penalties, and Parameter Continuation

Elsayed Mohamed

Abstract

This paper develops an exploratory continuous-relaxation heuristic for the Euclidean Traveling Salesperson Problem (TSP). The method represents undirected edges by symmetric weights in the interval [0, 1] and optimizes over a degree-constrained fractional 2-factor polytope. Three mechanisms are combined: a linear edge-cost objective, a spectral regularizer based on the Fiedler eigenvalue to encourage connectivity, and a quartic double-well penalty to encourage near-binary edge weights. Their weights are increased according to a parameter-continuation schedule. A projected-gradient prototype is provided, including an explicit projection onto the degree-constrained feasible set and a separate reconstruction stage that returns a valid Hamiltonian cycle using a weight-biased nearest-neighbour construction followed by 2-opt.

Several distinctions are important. The degree-constrained relaxation is not the Birkhoff–von Neumann polytope and is not the full TSP polytope: it permits fractional solutions and disconnected 2-factors. The spectral term encourages connectivity but does not enforce Hamiltonicity. The quartic penalty encourages, but does not guarantee, binary values. The original cost interpolation is not Gaussian convolution and does not preserve an optimizer in the claimed general sense. Accordingly, no global-convergence theorem, polynomial-time bound, or implication for P versus NP is claimed. The formulation is presented as a testable heuristic whose value must be established by implementation checks and benchmark experiments.

Keywords: Traveling Salesperson Problem, continuous relaxation, 2-factor polytope, graph Laplacian, Fiedler eigenvalue, projected gradient, continuation, 2-opt.

1. Introduction

The Euclidean TSP asks for a shortest closed tour that visits every city exactly once. A permutation-based enumeration examines factorially many tours, but this is not a description of every TSP algorithm: exact methods and heuristics use substantially more structure. The TSP is NP-hard, and a practical continuous relaxation should therefore be assessed as a heuristic unless a rigorous complexity result is established.

Continuous edge weights can be useful as an intermediate representation, but a fractional matrix is not itself a tour. Three obstacles are central to this formulation:

1. **Degree feasibility and integrality.** Continuous weights can satisfy degree equations without being binary.
2. **Disconnected 2-factors.** Even a binary, degree-two graph can be a union of several disjoint cycles rather than one Hamiltonian cycle.
3. **Non-convex regularization.** Spectral and binary penalties create a non-convex objective whose stationary points need not be globally optimal.

This paper makes the model explicit and provides a reproducible small-instance prototype. It does not present a proof of global optimality, polynomial-time solvability, or P = NP.

2. Variables and the Degree-Constrained Relaxation

Let the cities be V = {1, ..., n}, with Euclidean coordinates p_i in R^2. Let c_ij = $||p_i - p_j||_2$ be the distance between cities i and j. We use one variable x_ij for each undirected edge {i, j}, with i < j, and define x_ji = x_ij and x_ii = 0 when representing the variables as a matrix X.

The degree-constrained feasible set is

P_2 = {X : X = X^T, X_ii = 0, 0 ≤ X_ij ≤ 1, and Σ_(j≠i) X_ij = 2 for every i}.

The set is convex. Its uniform feasible point has X_ij = 2/(n - 1) for i ≠ j. The conditions enforce degree two in the weighted sense, but they do not force X to be binary.

This is a degree-constrained fractional 2-factor relaxation. It is **not** the Birkhoff–von Neumann polytope: that classical polytope describes doubly stochastic matrices with row and column sums equal to one. Nor is P_2 the full TSP polytope. Even an integral point satisfying degree two may consist of several cycles. Subtour-elimination constraints, for example Σ_(i<j, i,j∈S) X_ij ≤ |S| - 1 for proper subsets S with |S| ≥ 2, are needed to exclude subtours at the integer-tour level; the full continuous TSP formulation has further polyhedral subtleties.

For numerical stability, normalize the distances by c_max = max_(i<j) c_ij, assuming c_max > 0, and write a_ij = c_ij/c_max. The base linear edge-cost objective is

C(X) = Σ_(i<j) a_ij X_ij.

For an integral Hamiltonian tour, this objective is its length divided by c_max.

3. Corrected Four-Mechanism Formulation

The formulation uses cost normalization, a degree-constrained relaxation, a spectral connectivity regularizer, and a quartic binary penalty. These components are design choices, not established physical laws or guarantees.

3.1 Cost normalization, not measure-preserving convolution

A prior draft described the interpolation

$E_\sigma = (1 - \sigma) C + \sigma \bar{c}$

as Gaussian convolution and asserted mass/centroid preservation and preservation of the global optimum. Those statements do not follow from this formula. The interpolation is not a convolution operator, and it does not establish any Wasserstein or measure-preserving property.

There is also a useful exact observation. For X in P_2, summing all degree equations gives Σ_(i<j) X_ij = n. Therefore

$\Sigma_{i<j} [(1 - \sigma)a_{ij} + \sigma \bar{c}]X_{ij} = (1 - \sigma)C(X) + \sigma \bar{c}\, n.$

For 0 ≤ σ < 1, this is a positive scaling of C(X) plus a constant on P_2, so it has the same minimizers as C(X). At σ = 1, every feasible point has the same interpolated objective. Consequently, this interpolation cannot be presented as a demonstrated convexification or as a mechanism that preserves a unique optimum while removing local minima. The prototype instead normalizes costs and uses continuation only to schedule the strengths of the non-linear regularizers.

3.2 Degree constraints through the 2-factor relaxation

The degree equations are imposed directly by projecting every update back onto P_2. This avoids describing a heuristic dual update as an exact interior-point method. It also makes the distinction between satisfying degree constraints and obtaining a Hamiltonian cycle explicit.

Given a tentative edge-weight vector y, the projection is the convex quadratic program

$\Pi_{P_2}(y) = \operatorname{argmin}_z \tfrac{1}{2}||z - y||^2$ subject to A z = 2 and 0 ≤ z ≤ 1,

where A is the unsigned vertex-edge incidence matrix: each column has a one at each endpoint of its edge. The reference code solves this small projection problem numerically with SLSQP. That choice is convenient for a prototype, not an efficient large-scale projection algorithm.

3.3 Spectral connectivity regularization

Let $L(X) = D(X) - X$ be the weighted graph Laplacian, where $D_{ii} = \Sigma_j X_{ij}$. Let $\lambda_2(X)$ be the second-smallest eigenvalue of $L(X)$. For a nonnegative weighted graph, $\lambda_2 > 0$ if and only if its support graph is connected.

We use the soft connectivity penalty

$$\mathbf{R_{conn}(X) = \gamma / (\lambda_2(X) + \varepsilon),}$$

where $\gamma \geq 0$ is a continuation weight and $\varepsilon > 0$ prevents division by zero. Minimizing this term favours larger algebraic connectivity. Because ε is positive, this is a **soft penalty**, not an infinite barrier: it remains finite at disconnected graphs.

If λ_2 is a simple eigenvalue and u is a corresponding unit eigenvector, the derivative with respect to an undirected edge variable is

$$\mathbf{\partial R_{conn}/\partial X_{ij} = -\gamma (u_i - u_j)^2 / (\lambda_2 + \varepsilon)^2.}$$

This follows from $\partial\lambda_2/\partial X_{ij} = (u_i - u_j)^2$ for a simple Laplacian eigenvalue. If λ_2 is repeated, the eigenvalue is not generally differentiable in the ordinary sense; a subgradient or a smoothed spectral alternative is needed. The prototype skips this gradient when the eigengap is too small. That safeguard is approximate and does not resolve all nonsmooth cases.

Connectivity is not Hamiltonicity. A connected weighted graph with degree two in the weighted sense need not be a binary cycle. Conversely, a binary degree-two graph may be disconnected. The spectral term is therefore a bias, not a subtour-elimination proof.

3.4 Quartic double-well penalty

To favour values near zero or one, define

$$\mathbf{R_{bin}(X) = \mu \Sigma_{(i<j)} X_{ij}^2 (1 - X_{ij})^2,}$$

where $\mu \geq 0$. For a single variable x,

$$\mathbf{d[x^2(1 - x)^2]/dx = 2x(1 - x)(1 - 2x).}$$

Thus the gradient-descent contribution is the negative of this derivative. For $0 < x < 0.5$ it tends to reduce x; for $0.5 < x < 1$ it tends to increase x. The point $x = 0.5$ is stationary and unstable for the isolated double-well potential. Projection onto P_2 couples the edge variables, however, so the penalty does not guarantee that all entries become binary or that a binary result is a Hamiltonian tour.

3.5 Combined objective and projected update

At iteration k, use

$$\mathbf{F_k(X) = C(X) + \gamma_k R_0(X) + \mu_k B(X),}$$

where $R_0(X) = 1/(\lambda_2(X) + \varepsilon)$ and $B(X) = \Sigma_{(i<j)} X_{ij}^2 (1 - X_{ij})^2$. Equivalently, the spectral term is $\gamma_k R_0(X)$. The projected update is

$$\mathbf{X_{(k+1)} = \Pi_{P_2}[X_k - \eta_k \nabla F_k(X_k)].}$$

The reference implementation uses a finite schedule in which γ_k and μ_k increase smoothly. Because the regularized objective is non-convex, the eigenvalue term can be nonsmooth, and the objective changes with k, the update is not claimed to monotonically decrease a single fixed energy or to converge to the global optimum.

4. Reference Python Prototype

The following implementation is intended for small instances and auditability. It uses SciPy for numerical projection onto the degree-constrained set. Its continuous output is not a tour; the code therefore also includes a distinct reconstruction stage. The reconstruction forms a tour with a nearest-neighbour rule biased by the continuous weights and then improves it using 2-opt on the original Euclidean distances.

```
from __future__ import annotations

import numpy as np
from scipy.optimize import minimize


class AlgebraicRelaxationTSP:
    """Exploratory continuous relaxation for small Euclidean TSP instances.

    This is a research prototype, not a proven polynomial-time or
    globally optimal TSP algorithm. The matrix X is a fractional
    degree-two relaxation; reconstruction is performed separately.
    """

    def __init__(self, num_nodes: int = 12, seed: int = 42):
        if num_nodes < 3:
            raise ValueError("num_nodes must be at least 3")
        rng = np.random.default_rng(seed)
        self.n = num_nodes
        self.coords = rng.uniform(0.0, 1000.0, size=(self.n, 2))

        diff = self.coords[:, None, :] - self.coords[None, :, :]
        self.dist = np.sqrt(np.sum(diff * diff, axis=2))
        np.fill_diagonal(self.dist, 0.0)
        max_dist = float(np.max(self.dist))
        if max_dist <= 0.0:
            raise ValueError("All generated points coincide")
        self.cost = self.dist / max_dist

        self.edges = [(i, j) for i in range(self.n)
                      for j in range(i + 1, self.n)]
        self.m = len(self.edges)
        self.incidence = np.zeros((self.n, self.m), dtype=float)
        for e, (i, j) in enumerate(self.edges):
            self.incidence[i, e] = 1.0
            self.incidence[j, e] = 1.0
        self.edge_cost = np.array([self.cost[i, j]
                                   for i, j in self.edges])

        # Uniform feasible point: every vertex has weighted degree 2.
        self.x0 = np.full(self.m, 2.0 / (self.n - 1))

    def edge_vector_to_matrix(self, x: np.ndarray) -> np.ndarray:
        X = np.zeros((self.n, self.n), dtype=float)
        for value, (i, j) in zip(x, self.edges):
            X[i, j] = X[j, i] = value
        return X

    def project_degree_two(self, y: np.ndarray, warm_start: np.ndarray,
                           tolerance: float = 1e-6) -> np.ndarray:
        """Euclidean projection onto A x = 2, 0 <= x <= 1."""
        constraints = {
            "type": "eq",
            "fun": lambda z: self.incidence @ z - 2.0,
            "jac": lambda z: self.incidence,
        }
        result = minimize(
            fun=lambda z: 0.5 * float(np.dot(z - y, z - y)),
            x0=warm_start.copy(),
            jac=lambda z: z - y,
            method="SLSQP",
            bounds=[(0.0, 1.0)] * self.m,
            constraints=constraints,
            options={"ftol": 1e-9, "maxiter": 200, "disp": False},
        )
        z = np.asarray(result.x, dtype=float)
        feasible = (
            np.min(z) >= -tolerance
            and np.max(z) <= 1.0 + tolerance
            and np.max(np.abs(self.incidence @ z - 2.0)) <= 5 * tolerance
        )
        if not feasible:
```

```
        raise RuntimeError(
            "Degree-two projection failed; try fewer steps or inspect SciPy"
        )
    return np.clip(z, 0.0, 1.0)

def spectral_gradient(self, X: np.ndarray, gamma: float,
                      eigengap_tol: float = 1e-7,
                      epsilon: float = 1e-3) -> tuple[np.ndarray, float]:
    """Gradient of gamma/(lambda_2 + epsilon), when lambda_2 is simple.

    If the second eigenvalue is nearly repeated, return a zero gradient
    for this term at the current step rather than pretending its ordinary
    derivative is defined. This is only a basic numerical safeguard.
    """
    grad = np.zeros(self.m, dtype=float)
    lap = np.diag(np.sum(X, axis=1)) - X
    evals, evecs = np.linalg.eigh(lap)
    lam2 = max(0.0, float(evals[1]))
    gap_above = float(evals[2] - evals[1]) if self.n > 2 else float("inf")
    gap_below = float(evals[1] - evals[0])
    if gamma == 0.0 or gap_above <= eigengap_tol or gap_below <= eigengap_tol:
        return grad, lam2
    u = evecs[:, 1]
    scale = -gamma / (lam2 + epsilon) ** 2
    for e, (i, j) in enumerate(self.edges):
        grad[e] = scale * (u[i] - u[j]) ** 2
    return grad, lam2

def solve(self, steps: int = 80, learning_rate: float = 0.05,
          gamma_max: float = 0.15, mu_max: float = 0.5,
          epsilon: float = 1e-3) -> np.ndarray:
    if steps < 1:
        raise ValueError("steps must be positive")
    x = self.x0.copy()
    for k in range(steps):
        t = k / max(1, steps - 1)
        gamma = gamma_max * t * t
        mu = mu_max * t * t
        X = self.edge_vector_to_matrix(x)

        grad_spectral, _ = self.spectral_gradient(
            X, gamma=gamma, epsilon=epsilon
        )
        grad_binary = 2.0 * mu * x * (1.0 - x) * (1.0 - 2.0 * x)
        grad = self.edge_cost + grad_spectral + grad_binary

        proposal = x - learning_rate * grad
        x = self.project_degree_two(proposal, warm_start=x)

    return self.edge_vector_to_matrix(x)

def tour_length(self, tour: list[int]) -> float:
    return float(sum(
        self.dist[tour[k], tour[k + 1]]
        for k in range(len(tour) - 1)
    ))

def reconstruct_tour(self, X: np.ndarray,
                     weight_bias: float = 0.15) -> list[int]:
    """Build a valid tour from X, then improve it with Euclidean 2-opt."""
    adjusted = self.cost - weight_bias * X
    np.fill_diagonal(adjusted, np.inf)

    unvisited = set(range(1, self.n))
    tour = [0]
    while unvisited:
        current = tour[-1]
        nxt = min(unvisited, key=lambda j: adjusted[current, j])
        tour.append(nxt)
        unvisited.remove(nxt)
    tour.append(0)
```

```
    # First-improvement 2-opt using the original Euclidean distances.
    improved = True
    while improved:
        improved = False
        for i in range(1, self.n - 1):
            for j in range(i + 1, self.n):
                a, b = tour[i - 1], tour[i]
                c, d = tour[j], tour[j + 1]
                delta = (self.dist[a, c] + self.dist[b, d]
                         - self.dist[a, b] - self.dist[c, d])
                if delta < -1e-9:
                    tour[i:j + 1] = reversed(tour[i:j + 1])
                    improved = True
                    break
            if improved:
                break
    return tour


if __name__ == "__main__":
    solver = AlgebraicRelaxationTSP(num_nodes=12, seed=42)
    X = solver.solve(steps=80)
    tour = solver.reconstruct_tour(X)
    assert len(tour) == solver.n + 1 and tour[0] == tour[-1]
    assert set(tour[:-1]) == set(range(solver.n))
    print("Maximum weighted-degree error:",
          float(np.max(np.abs(X.sum(axis=1) - 2.0))))
    print("Tour:", tour)
    print("Tour length:", solver.tour_length(tour))
```

4.1 Implementation notes

- The projection enforces row/weighted-degree sums of two and the box constraints to numerical tolerance; it is not the Sinkhorn–Knopp algorithm.
- The spectral derivative is used only when the Fiedler eigenvalue appears simple. The code's zero-gradient fallback is a practical safeguard, not a mathematically complete subgradient method.
- The quartic penalty has the correct gradient sign for minimization. It is not an exact integrality constraint.
- The tour-reconstruction stage always constructs a permutation tour and applies 2-opt using the original distances. It does not prove optimality.
- Computing a full eigendecomposition costs $O(n^3)$ per iteration. The SLSQP projection can also be costly, and no large-scale runtime claim is made.

5. What the Mathematics Does and Does Not Establish

5.1 Feasibility is not integrality

The constraints defining P_2 form a convex set, and the uniform initial matrix belongs to it. Projection gives a degree-feasible fractional matrix (up to solver tolerance). It does not establish that the solution is a binary 2-factor or a Hamiltonian cycle.

5.2 Connectivity is not Hamiltonicity

For nonnegative weights, $\lambda_2 > 0$ characterizes connectivity of the support graph. It does not force each entry to be binary, and it does not by itself impose the combinatorial conditions of a Hamiltonian cycle. The spectral regularizer can encourage connectivity but cannot replace subtour constraints or tour reconstruction.

5.3 No Lyapunov or homotopy theorem is asserted

A proof of convergence would require specifying a fixed objective or a controlled time-varying system, handling eigenvalue multiplicities, proving sufficient regularity, and analysing the projection/update rule. A proof of global

optimality would require additional arguments not present here. Since the objective includes a non-convex double-well penalty and a spectral term, stationary points may be local or degenerate.

5.4 Complexity accounting

For K iterations, each dense symmetric eigendecomposition is $O(n^3)$. Cost-gradient calculations are $O(n^2)$. The reference implementation also solves a constrained quadratic projection numerically at each iteration; its total cost depends on the numerical solver and its convergence. Without a bound on K and on the projection work, no polynomial-time bound for the complete method follows. In particular, an $O(n^5)$ claim is not justified by simply multiplying a per-step estimate by an assumed iteration count.

6. Evaluation Protocol

A meaningful evaluation should compare the complete pipeline—not just its fractional matrix—with established baselines on standard Euclidean TSP instances. At minimum:

1. Use published TSPLIB instances and also seeded random Euclidean instances over increasing n.
2. Report tour length, optimality gap when a known optimum is available, runtime, success/failure counts, and variation across seeds.
3. Compare against nearest neighbour, nearest neighbour + 2-opt, and a strong solver such as LKH or OR-Tools where available.
4. Run ablations with $\gamma = 0$ (no spectral term), $\mu = 0$ (no binary penalty), and both terms disabled.
5. Check every output: the tour must visit each city exactly once before returning to the start; the continuous matrix must be symmetric, have zero diagonal, respect [0,1] bounds, and satisfy degree sums within stated tolerance.
6. Report parameter choices and all instances, including failures. Do not infer polynomial-time behaviour from a small finite benchmark.

7. Limitations and Open Questions

The method is exploratory. Its core assumptions and open questions are:

- Does the spectral term improve the reconstructed tour compared with the same reconstruction without spectral weights?
- Does the quartic penalty improve reconstruction quality, or does the degree projection counteract its binarizing tendency?
- How sensitive are results to γ, μ, the learning rate, and the number of iterations?
- Can the projection and the treatment of repeated Fiedler eigenvalues be made both reliable and scalable?
- Would explicit subtour constraints or stronger 2-factor methods outperform a spectral penalty?
- How does this complete pipeline compare with established TSP heuristics on standard benchmarks?

The framework should be described as a continuous-relaxation heuristic, not as a convex solver, an interior-point algorithm, a proven polynomial-time algorithm, or a resolution of the P-versus-NP question.

8. Conclusion

This paper specifies a degree-constrained algebraic relaxation with spectral connectivity regularization and a quartic binary penalty. It corrects three common category errors: the degree-two relaxation is not the Birkhoff–von Neumann polytope; linear interpolation toward mean costs is not Gaussian, measure-preserving smoothing; and connectivity regularization does not guarantee Hamiltonicity. A small reference implementation is provided, with a numerical projection and a separate tour-reconstruction routine.

The scientific contribution at this stage is a testable formulation and an auditable prototype. Its practical value remains an empirical question. Future work should prioritize validation, ablation studies, stronger projection methods, and

benchmarking against established solvers. No claim of global convergence, polynomial-time complexity, or P = NP is made.

References

- Aarts, E., and Lenstra, J. K., eds. *Local Search in Combinatorial Optimization*. Wiley, 1997.
- Applegate, D. L., Bixby, R. E., Chvátal, V., and Cook, W. J. *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, 2006.
- Boyd, S., and Vandenberghe, L. *Convex Optimization*. Cambridge University Press, 2004.
- Fiedler, M. "Algebraic Connectivity of Graphs." *Czechoslovak Mathematical Journal*, 23(2), 1973, 298–305.
- Sinkhorn, R., and Knopp, P. "Concerning Nonnegative Matrices and Doubly Stochastic Matrices." *Pacific Journal of Mathematics*, 21(2), 1967, 343–348.
- Tero, A., et al. "Rules for Biologically Inspired Adaptive Network Design." *Science*, 327(5964), 2010, 439–442. (Background for the biological-network analogy, not a justification of the present equations.)